

Consistency for size() functions

Document #: P1970
Date: 2019-11-07
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Hannes Hauswedell
<h2@fsfe.org>

Contents

- [1 Introduction](#)
- [2 Motivation and Scope](#)
 - [2.1 There is no ssize that works for all ranges](#)
 - [2.2 std::\[s\]size should behave as std::ranges::\[s\]size](#)
- [3 Impact on the standard](#)
- [4 Design decisions](#)
- [5 Wording](#)

§ 1 Introduction

C++17 introduced `std::size()` and C++20 will introduce `std::ssize()` and `std::ranges::size()`. Why do we need three and do they solve all problems in regard to size? After DE269 raised these questions, LEWG requested I write this paper.

§ 2 Motivation and Scope

	<code>.size()</code>	<code>.size()</code> or <code>end() - begin()</code>
unsigned †	<code>std::size()</code>	<code>std::ranges::size()</code>
signed	<code>std::ssize()</code>	<i>none</i>

† This can be signed under certain circumstances, but is not for any types in the standard library. Criticism of this was raised in LEWG, but this is independent of everything else and not touched by this paper to reduce last-minute impact.

`std::ranges::size()` was introduced (also) because it was desired that subtractable iterator-sentinel-pairs should be sufficient for a range to qualify it as a

`std::ranges::sized_range`. This means `std::size()` will not work on certain sized ranges, but `std::ranges::size()` will.

`std::ssize()` has been introduced (with some controversy), because there was the desire to have a signed size type that can be used (among other things) in loops and comparisons with signed counter variables. It is defined as `std::size()` but casts the type to a comparable signed type. This solves the problem but only works on the subset of sized ranges that `std::size()` works on.

§ 2.1 There is no ssize that works for all ranges

The first problem and original intent of DE269 is that there is no signed size function that works on all sized ranges, i.e. if the arguments for adding `std::ssize()` are valid, they imply that we also need `std::ranges::ssize()` with the semantics of `std::ranges::size()` and a cast to a signed type.

The current state is inconsistent in regard to signed vs unsigned (inside `std::ranges::`).

Solution: add `std::ranges::ssize()`

§ 2.2 `std::[s]size` should behave as `std::ranges::[s]size`

During discussion in LEWG it was criticised that we have different semantics in the different namespaces at all, i.e. that **the current state is inconsistent in regard to `std::` vs `std::ranges::`**. Because the types that `std::size()` applies to is a subset of the types that `std::ranges::size()` applies to, it was proposed to make them behave the same or even just have two functions instead of three or four.

It also became evident that this subsumption is only true now (before C++20), because, after C++20, ranges can opt-out of `std::ranges::size()` via `disable_sized_range` – but not out of `std::size()`. Thus any fix in this area must happen now.

The most obvious solution would be to have `std::size()` behave like `std::ranges::size()`, remove the latter, and have `std::ssize()` refer to `std::size()`. This is not possible, however, because `std::ranges::size()` is a function object and `std::size()` is a function that is subject to ADL. Replacing the function with a function object will break code that relied on ADL.

Solution: `std::size()` and `std::ssize()` are defined as functions that call their counterparts in `std::ranges::`. This guarantees that if we must have more than two interfaces, at least we only have two different semantics clearly denoted by name (signed vs unsigned).

§ 3 Impact on the standard

See above why these changes would be breaking after C++20.

See below for wording.

§ 4 Design decisions

This is the current proposal:

.size() or end() - begin()	
unsigned	<code>std::size()</code> , <code>std::ranges::size()</code>
signed	<code>std::ssize()</code> , <code>std::ranges::ssize()</code>

The discussion in LEWG also proposed the following other “solutions”:

1. **Replace `std::[s]size` with `std::ranges::[s]size`.** This doesn’t work because of function VS function object, see above.
2. **Only add `std::ranges::ssize()`, don’t touch `std::size()` and `std::ssize()`.**
Less invasive, but only fixes the first inconsistency.
3. **Don’t add `std::ranges::ssize()` and remove `std::ssize()`.** This creates some consistency with regard to the absence of any signed size function. It doesn’t solve the second inconsistency above.
4. **Don’t add `std::ranges::ssize()`, but re-define `std::ssize()` in terms of `std::ranges::size()` instead of `std::size`.** It solves the problem of a lacking `ssize` for all ranges. However, it increases inconsistency within `std::`. And it does not solve the second inconsistency.

If there is no consensus to accept this proposal as a whole, I would still suggest doing option 2. Option3 would be possible, too, but is likely to decrease consensus in plenary.

§ 5 Wording

§23.07

```
-template<class C> constexpr auto size(const C& c) -> decltype(c.size());  
- Returns: c.size().  
-  
-template<class T, size_t N> constexpr size_t size(const T (&array)[N])  
  noexcept;  
- Returns: N.  
  
+template <class C> constexpr auto size(C&& c)  
+ -> decltype(std::ranges::size(std::forward<C>(c)));  
+ Returns: std::ranges::size(std::forward<C>(c)).  
  
-template<class C> constexpr auto ssize(const C& c)
```

```

-   -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
-   Returns: static_cast<common_type_t<ptrdiff_t,
-           make_signed_t<decltype(c.size())>>>(c.size())
-
- template<class T, ptrdiff_t N> constexpr ptrdiff_t ssize(const T (&array)
-     [N]) noexcept;
- Returns: N.

+template <class C> constexpr auto ssize(C&& c)
+ -> decltype(std::ranges::ssize(std::forward<C>(c)));
+ Returns: std::ranges::ssize(std::forward<C>(c)).

```

Introduce new § 24.3.10 after §24.3.9 `ranges::size`

```

+24.3.10 ranges::ssize
+
+The name ssize denotes a customization point
+  object([customization.point.object]). The
+expression ranges::ssize(E) for some subexpression E with type T is
+  expression-equivalent to:
+ - static_cast<common_type_t<ptrdiff_t,
+   make_signed_t<decltype(ranges::size(E))>>>(ranges::size(E))
+   if that is valid.
+ - Otherwise, ranges::ssize(E) is ill-formed.

```